

Lab 2: Routy - A Small Routing Protocol

Sevastian Bajenov

September 7, 2026

1 Introduction

In this laboratory work, my main goal was to implement link-state routing protocol. It is the most used protocol for Internet routers which is based on informing neighbors about your direct links rather than sending complete routing tables to them as in the distance vector approach. After completing the implementation part I had the opportunity to test the whole routing setup and verify how the status of each router changes depending on the network events.

2 Main problems and solutions

The router developed in the context of this laboratory work is based on several important entities, such as history, interfaces, network map and routing table. The main challenge for me was to understand how they were interconnected.

Routing table and map are the most crucial components of a router. The map represents the network topology and contains all nodes with their direct connections. Routing table is constructed using the Dijkstra algorithm and has tuples of values in the form node, gateway. Each router also has a list of the interfaces (basically the information about the other router processes) and its own history to distinguish new messages from the old ones.

One of the challenges was to understand how to handle the actual routing of messages because, for instance, when the message reaches the destination the router should correctly handle it. I decided to stick to the original implementation with two clauses in order to avoid introducing complex logic in handling lookup of the router itself.

```
{route, Name, _, Message} ->
    io:format("~w: received message ~w ~n", [Name, Message]),
    router(Name, N, Hist, Intf, Table, Map);
{route, To, From, Message} ->
    io:format("~w: routing message (~w)~n", [Name, Message]),
```

```

    case dijkstra:route(To, Table) of
      {ok, Gateway} ->
        case interface:lookup(Gateway, Intf) of
          {ok, Pid} ->
            Pid ! {route, To, From, Message};
          notfound ->
            ok
        end;
      notfound ->
        ok
    end,

    router(Name, N, Hist, Intf, Table, Map);

```

Another problem that caught my eye was that the map of the network does not reflect the latest changes in case a node fails. The routers invoke broadcasting and update their routing tables (being manually instructed to do so) but the map stays with the "dead" entry. This does not influence the algorithm but the map can accumulate such entries over time.

3 Evaluation

In order to evaluate the routing protocol I developed a script which creates a ring-like network of routers:

```

routers() ->
  [{r1, stockholm}, {r2, gothenburg}, {r3, malmo}, {r4, uppsala}, {r5, lund}]

links() ->
  [{r1, r2}, {r2, r3}, {r3, r4}, {r4, r5}, {r5, r1}].

```

The script initiates the router connections, broadcasts link-state messages, triggers routing-table updates, simulates node failure and message passing. Below are presented pieces of the output of the test script:

```

=== computing routing tables ===
Name: stockholm
Sequence number: 2
History: [{lund,1},{gothenburg,1},{malmo,1},{uppsala,1},{stockholm,inf}]
Interfaces: [{lund,#Ref<0.1661209752.1228406797.180121>,<0.88.0>},
{gothenburg,#Ref<0.1661209752.1228406797.180120>,<0.85.0>}]
Routing table: [{malmo,gothenburg},{stockholm,gothenburg},{uppsala,lund},
{lund,lund},{gothenburg,gothenburg}]

```

```

Map: [{lund,[stockholm,uppsala]},{gothenburg,[malmo,stockholm]},
{malmo,[uppsala,gothenburg]},{uppsala,[lund,malmo]}]

=== routing stockholm -> malmo (expect 2 hops via gothenburg) ===
stockholm: routing message (hello_from_stockholm)
gothenburg: routing message (hello_from_stockholm)
malmo: received message hello_from_stockholm

=== routing uppsala -> stockholm (expect 2 hops via lund) ===
uppsala: routing message (hello_from_uppsala)
lund: routing message (hello_from_uppsala)
stockholm: received message hello_from_uppsala

=== simulating link failure: stopping gothenburg ===
stockholm: exit received from gothenburg
malmo: exit received from gothenburg

=== survivors re-broadcast (x2) and update after the failure ===

=== routing stockholm -> malmo again (expect reroute via lund, uppsala) ===
stockholm: routing message (hello_after_failure)
lund: routing message (hello_after_failure)
uppsala: routing message (hello_after_failure)
malmo: received message hello_after_failure

```

The routers handle the network events correctly according to the output and manage to route the messages using the most efficient route.

4 Conclusions

In this laboratory work I managed to implement a simple routing protocol based on link-state messages and tested it under different conditions. One of the possible improvements considers stale map condition which was mentioned earlier. The map should be updated when the node sends a DOWN message and disconnects. Another suggestion would be related to the history management because in case of node failure and reconnection its sequence number is going to be reinitialized as well. Finally, the update and broadcast actions should not be manual but instead be related to the link-state and node failure message reception.